

TED UNIVERSITY

CMPE 492/SENG492— SENIOR PROJECT II

Low-Level Design Report Spring 2026



LLM Valley
<https://llmvalley.framer.website/>

Faculty of Engineering

LLM Valley

Low Level Design Report

TEAM 21

March 2026

Project Team Members:

Yusuf Sadi Pesen , Game Designer and Developer

Reyhan Sena Çimen, Game Developer and 2D Artist

Egemen Asım Ersoy, Game Developer

Deniz Şeker, Game Developer

Supervisor:

Eren Ulu

Jury Members:

Ulaş Güleç

Ayşenur Birtürk

Coordinator:

Prof. Dr. Gökçe Nur Yılmaz

Table of Contents

Table of Contents.....	3
Table of Tables.....	5
Table of Figures.....	6
Related Documents	7
Change History.....	8
1. Introduction and Scope.....	9
1.1 Object-Design Trade-offs.....	9
1.2 Interface Documentation Guidelines	10
1.3 Engineering Standards — UML Models.....	10
1.4 Definitions, Acronyms, and Abbreviations	11
2. Packages.....	12
2.1 Package Overview — Figure 1.....	12
2.2 Core Game Package — Figure 2	12
2.3 AI & Dialogue Package — Figure 3.....	13
2.4 Player & Interaction Package — Figure 4.....	13
2.5 Economy & Environment Package — Figure 5	13
2.6 UI Package — Figure 6	13
3. Class Interfaces.....	14
3.1 Core Game Package	14
3.1.1 GameManager	14
3.1.2 DayNightCycle.....	15
3.1.3 SaveSystem.....	16
3.2 AI & Dialogue Package.....	17
3.2.1 LLMDialogueSystem.....	17
3.2.2 LLMApiClient.....	18
3.2.3 NPCManager.....	19
3.2.4 NPC	19
3.2.5 NPCStats.....	20
3.3 Player & Interaction Package.....	22
3.3.1 Player.....	22
3.3.2 InputManager	23
3.3.3 Inventory	23
3.3.4 PlayerStats	24
3.3.5 Item.....	24

3.4 Economy & Environment Package.....	25
3.4.1 FarmingSystem.....	25
3.4.2 CropPlot.....	25
3.4.3 Crop.....	26
3.4.4 TradingSystem	26
3.5 UI Package.....	28
3.5.1 UIManager	28
3.5.2 DialogueUI.....	29
4. Glossary	30
5. References	32

Table of Tables

Table 1: Related Documents

Table 2: Change History

Table 3: Object-Design Trade-offs

Table 4: GameManager — Public Interface

Table 5: DayNightCycle — Public Interface

Table 6: SaveSystem — Public Interface

Table 7: LLMDialogueSystem — Public Interface

Table 8: LLMApiClient — Public Interface

Table 9: NPCManager — Public Interface

Table 10: NPC — Public Interface

Table 11: Player — Public Interface

Table 12: InputManager — Public Interface

Table 13: Inventory — Public Interface

Table 14: PlayerStats — Public Interface

Table 15: FarmingSystem — Public Interface

Table 16: CropPlot — Public Interface

Table 17: TradingSystem — Public Interface

Table 18: UIManager — Public Interface

Table 19: DialogueUI — Public Interface

Table 20: Glossary Terms

Related Documents

Name	Description	URL / Location
Project Proposal	Initial project scope, objectives, and team plan	Proposal-Group_21-LLM_Valley-31_10_2025.pdf
Project Analysis Report	Functional/non-functional requirements, use cases, UML models	Project_Analysis_Report-Group_21-LLM_Valley-28_11_2025.pdf
Project Specifications Report	System features, interface requirements, constraints	CMPE491_Project_Specifications_Report_Team21_LLMValley.pdf
High Level Design Report	Architecture overview, subsystem decomposition, hardware/software mapping	High_Level_Design_Report_Group21_LLMValley.pdf
LLM Valley Website	Public-facing project website	https://llmvalley.framer.website/
Low Level Design Version 1	Specifies the detailed components and interactions needed for implementation.	LLMValley_LowLevelDesign_Report CMPE 492 - CMPE 492.pdf

Change History

Date	Changed By	Change Description
31.10.2025	Team 21	Initial project proposal submitted; scope and objectives defined.
28.11.2025	Team 21	Project Analysis Report completed; UML models, use cases, and dynamic models added.
14.11.2025	Team 21	Project Specifications Report (v1) submitted with feature specs and requirements.
30.12.2025	Team 21	High Level Design Report completed; client-server architecture and subsystems defined.
20 March 2026	Team 21	Low Level Design Report authored; detailed class interfaces, package diagrams, and method-level specifications added.
27 March 2026	Team 21	Low Level Design Report authored; detailed class interfaces, package diagrams, UML diagrams, and method-level specifications added.

1. Introduction and Scope

This Low-Level Design (LLD) Report provides the detailed technical specification for LLM Valley, a narrative-driven life simulation demo developed as part of the CMPE 491/SENG491 Senior Project I at TED University. LLM Valley integrates Large Language Models (LLMs) into real-time non-player character (NPC) dialogue systems, replacing conventional static dialogue trees with dynamic, context-aware conversations.

The purpose of this document is to bridge the gap between the high-level architectural decisions documented in the High-Level Design (HLD) Report and the actual implementation. It provides every class's precise interface — including method signatures, field definitions, pre-conditions, post-conditions, and exception handling contracts — as well as formal package decomposition and UML diagrams sufficient for developers to write code without ambiguity.

The scope of this document covers all five subsystem packages identified in HLD: Core Game, AI & Dialogue, Player & Interaction, Economy & Environment, and UI. It is intended for the development team, the project supervisor, and jury members evaluating the project's technical rigor.

1.1 Object-Design Trade-offs

Several key design trade-offs were made during the object-level design phase. These decisions balance competing quality attributes such as performance, maintainability, and simplicity.

Trade-off	Option A (Chosen)	Option B (Rejected)	Rationale
LLM Response Handling	Asynchronous coroutines (async/await in C#)	Synchronous blocking calls	Blocking the Unity main thread for 3–5 seconds would freeze all rendering, input, and physics. Coroutines allow the game loop to continue while awaiting the HTTP response.
Dialogue State Management	Finite State Machine (FSM) on NPC component	Simple boolean flags on NPCManager	An FSM provides a clear, extensible model for Idle → InputListening → Processing → ResponseGeneration → Termination transitions, preventing illegal state combinations.
Stat Update Mechanism	Keyword-based pattern matching on LLM response	Full sentiment analysis via secondary API call	A secondary API call would double latency. Keyword matching (e.g., detecting '[Trust+1]' tags embedded in LLM output) is deterministic and adds zero network overhead.
Save/Load Persistence	Local JSON file via Unity JsonUtility	SQL or NoSQL cloud database	The demo is single-player with no online requirement. JSON is zero-dependency, human-readable for debugging, and fully sufficient for saving day count, inventory, and stats.
LLM Provider Coupling	Abstracted LLMApiClient interface with concrete DeepSeek/Gemini implementations	Hard-coded DeepSeek calls	Decoupling via an interface allows hot-swapping of LLM providers without refactoring business logic, directly satisfying the Modularity non-functional requirement.

Prompt Construction	Runtime string concatenation (npc.role + context + player input)	Pre-authored static prompts per NPC	Runtime construction allows the prompt to embed dynamic game state (day count, relationship stats, inventory) giving the LLM the context it needs for coherent, relevant responses.
NPC Roster Management	NPCManager holds List<NPC> and iterates for stat updates	Individual NPC objects self-manage	Centralised management in NPCManager makes it easy to add new NPCs, broadcast global events (e.g., end-of-day), and perform batch saves without coupling NPC objects to each other.

1.2 Interface Documentation Guidelines

All class interfaces documented in Section 3 follow these conventions:

- Method names are in camelCase (C# Unity convention).
- Fields are listed with their type, access modifier (public / private / protected), and a brief description.
- Method entries include: signature, parameters, return type, pre-conditions, post-conditions, and exceptions/error states.
- Unity-specific lifecycle methods (Awake, Start, Update, OnApplicationQuit) are documented where they carry significant logic.
- Async methods are marked with the async keyword and their coroutine/Task return type.
- All parameters include type and name; optional parameters note their default value.
- Cross-references to other classes are hyperlinked via section numbers.

1.3 Engineering Standards — UML Models

This document uses UML 2.5 notation for all diagrams. The following diagram types are employed:

- Class Diagrams — Show class structure, fields, methods, and relationships (association, aggregation, composition, dependency). Multiplicity is annotated on association ends.
- Sequence Diagrams — Show the temporal ordering of method calls between objects for key interaction scenarios. Lifelines use the format `objectName : ClassName`.
- State Machine Diagrams — Show the FSM governing the dialogue system's control flow.
- Package Diagrams — Show the logical grouping of classes into packages and inter-package dependencies.

All diagrams referenced by figure number in this document are provided inline as described. Due to tool constraints, detailed UML diagrams are described textually alongside the class interfaces; companion vector diagrams are maintained in the project's version-controlled repository.

1.4 Definitions, Acronyms, and Abbreviations

Term	Definition
LLM	Large Language Model — an AI model (e.g., DeepSeek, Google Gemini) that generates human-like text given a prompt.
NPC	Non-Player Character — a game character controlled by the AI system rather than the player.
API	Application Programming Interface — the HTTPS endpoint exposed by the LLM cloud provider.
FSM	Finite State Machine — a computational model describing states and transitions; used for the dialogue control flow.
HLD	High Level Design — the architectural design document produced in the previous project phase.
LLD	Low Level Design — this document; provides detailed, implementation-ready specifications.
JSON	JavaScript Object Notation — the data serialization format used for save files and API payloads.
DXA	Document eXtended Attribute — unit of measurement used in OOXML documents (1440 DXA = 1 inch).
Prefab	Unity's reusable, pre-configured GameObject template.
MonoBehaviour	Unity's base class for scripts attached to GameObjects.
Coroutine	Unity's cooperative multitasking mechanism for executing code across multiple frames.
GDD	Game Design Document — the reference document describing gameplay mechanics and narrative.
CRUD	Create, Read, Update, Delete — basic data operations performed on save-file entities.
REST	Representational State Transfer — the architectural style used by the LLM API endpoints.
UI	User Interface — visual elements through which the player interacts with the game.
PR	Pseudo-Requirement — a design guideline not formally verified but influencing UX decisions.

2. Packages

The LLM Valley codebase is organized into five top-level packages (Unity assembly-level namespaces). Each package encapsulates a cohesive set of responsibilities and minimizes coupling to other packages. All inter-package communication flows through well-defined interfaces, primarily through events raised by the GameManager or direct references injected at initialization time.

2.1 Package Overview — Figure 1

Figure 1: LLM Valley Complete System Package Diagram

The five packages are:

Package Name	C# Namespace	Responsibility Summary	Key Classes
Core Game	LLMValley.Core	Main game loop, global state, time management, persistence.	GameManager, DayNightCycle, SaveSystem
AI & Dialogue	LLMValley.AI	LLM API integration, NPC dialogue FSM, prompt engineering, stat keyword analysis.	LLMDialogueSystem, LLMApiClient, NPCManager, NPC, NPCStats
Player & Interaction	LLMValley.Player	Player input capture, character movement, inventory management, player statistics.	Player, InputManager, Inventory, PlayerStats, Item
Economy & Environment	LLMValley.Economy	Crop lifecycle, farming grid, shop transactions, dynamic pricing.	FarmingSystem, CropPlot, Crop, TradingSystem
UI	LLMValley.UI	All on-screen rendering; stat bars, chat history, shop overlay, notification messages.	UIManager, DialogueUI

2.2 Core Game Package — Figure 2

The Core Game package acts as the root of the application. GameManager is implemented as a Singleton (one instance persists across scenes). It holds a reference to SaveSystem and coordinates all other subsystem managers. DayNightCycle fires the OnDayAdvanced event at end-of-day, which GameManager handles to orchestrate crop growth, stat refreshes, and save operations. SaveSystem serializes a GameSaveData value-object to disk.

- GameManager → has-one DayNightCycle (composition, 1:1)
- GameManager → has-one SaveSystem (composition, 1:1)
- GameManager → manages-many NPCManager, FarmingSystem, UIManager, InputManager (associations, 1:1 each)

2.3 AI & Dialogue Package — Figure 3

This package is the core innovation of LLM Valley. LLMDialogueSystem orchestrates prompt construction and API calls. LLMApiClient is the sole class that holds the API key and the HTTPS endpoint string, ensuring secrets are not scattered. NPCManager maintains the roster (List<NPC>) and routes incoming player messages to the correct NPC instance. Each NPC holds its NPCStats value-object and its current dialogue FSM state.

- NPCManager → manages-many NPC (1:N)
- NPC → composed-of NPCStats (1:1)
- LLMDialogueSystem → depends-on LLMApiClient (1:1)
- NPC → uses LLMDialogueSystem (dependency)

2.4 Player & Interaction Package — Figure 4

Player is the central class. It holds references to Inventory and PlayerStats. InputManager is an independent component that fires input events; Player subscribes to relevant events. Inventory holds a List<Item>. PlayerStats holds numeric fields (mental, money) and mutation methods.

- Player → composed-of Inventory (1:1)
- Player → composed-of PlayerStats (1:1)
- Inventory → contains-many Item (1:N)

2.5 Economy & Environment Package — Figure 5

FarmingSystem manages an array of CropPlot references. Each CropPlot optionally contains one Crop. The Crop value-object holds cropType, growthStage, growthTime, and sellPrice. TradingSystem holds a price dictionary and calls Inventory and PlayerStats mutation methods to execute transactions.

- FarmingSystem → manages-many CropPlot (1:N)
- CropPlot → contains-one Crop (0..1:1, optional)
- TradingSystem → interacts-with Inventory and PlayerStats (dependencies)

2.6 UI Package — Figure 6

UIManager is the single point of contact for all rendering updates. It holds a reference to DialogueUI for chat-history rendering. Other packages call UIManager.ShowNotification() or UIManager.UpdateStatBars() rather than manipulating Unity UI components directly.

- UIManager → manages DialogueUI (1:1)
- UIManager → subscribes to GameManager events (dependency)

3. Class Interfaces

This section documents the precise interface contract for every class in LLM Valley, organized by package. For each class the following sub-sections are provided: (a) Class Overview, (b) Fields, (c) Constructor / Unity Lifecycle, (d) Public Methods, and (e) Private / Internal Methods relevant to understanding the design.

3.1 Core Game Package

3.1.1 GameManager

GameManager is the application entry point and global coordinator. It is implemented as a Unity Singleton: only one instance may exist at runtime, and it persists across scene loads via DontDestroyOnLoad.

Fields:

Field	Type	Access	Description
instance	GameManager	public static	Singleton reference. Assigned in Awake(). Null before initialization.
currentDay	int	public	Current in-game day number. Starts at 1. Incremented by EndDay().
saveSlot	SaveSystem	private	Reference to the SaveSystem component on this GameObject.
npcManager	NPCManager	private	Reference to the NPCManager in the scene.
farmingSystem	FarmingSystem	private	Reference to the FarmingSystem in the scene.
uiManager	UIManager	private	Reference to the UIManager Canvas object.
inputManager	InputManager	private	Reference to the InputManager component.
currentState	GameState	private	Enum {MainMenu, Playing, Paused, Sleeping, GameOver}. Current application state.

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
void Awake()	Scene loaded.	Singleton instance set; DontDestroyOnLoad applied; saveSlot, npcManager, farmingSystem, uiManager, inputManager resolved via GetComponent / FindObjectOfType.	Logs error and destroys duplicate if instance != null.
void Start()	Awake() completed.	Calls saveSlot.Load(). If save data exists, applies loaded state to currentDay, NPCManager,	IOException logged; new game started on failure.

		FarmingSystem. Calls StartGame().	
void StartGame()	All subsystem references valid.	Sets currentState = GameState.Playing. Enables InputManager. Fires OnGameStarted event.	None.
void EndDay()	currentState == GameState.Playing.	Increments currentDay. Calls farmingSystem.AdvanceGrowth(). Calls npcManager.OnDayEnd(). Calls saveSlot.Save(). Sets currentState = GameState.Sleeping briefly, then back to Playing.	None expected; logs warning if farmingSystem is null.
void ChangeState(GameState newState)	newState is a valid GameState enum value.	Sets currentState = newState. Fires OnStateChanged(newState) event.	ArgumentException if newState is out of range.
void OnApplicationQuit()	Application is closing.	Calls saveSlot.Save() to persist final game state.	IOException logged; state may not persist on hard crash.

3.1.2 DayNightCycle

DayNightCycle manages in-game time. It tracks fractional time within a day and exposes a boolean IsNight() check used by NPCs and the UI to modify dialogue tone and visual state.

Fields:

Field	Type	Access	Description
currentTime	float	public	Fraction of the day elapsed (0.0 = dawn, 1.0 = midnight). Updated every frame in Update().
dayLengthSeconds	float	private	Real-world seconds that constitute one in-game day. Default: 300.0f (5 minutes).
nightThreshold	float	private	currentTime value above which IsNight() returns true. Default: 0.75f.

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
void Update()	MonoBehaviour active.	currentTime += Time.deltaTime / dayLengthSeconds. Fires OnNightfall event once per day when currentTime crosses nightThreshold.	None.
void UpdateTime()	Called by GameManager.EndDay().	Resets currentTime to 0.0f for the new day.	None.
bool IsNight()	None.	Returns currentTime >= nightThreshold.	None.

3.1.3 SaveSystem

SaveSystem handles serialization of all persistent game data to a local JSON file. The file is stored at `Application.persistentDataPath + '/save.json'`.

Fields:

Field	Type	Access	Description
saveFilePath	string	private	Full file path. Computed once in Awake().

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
void Save()	GameManager.instance is not null.	Constructs GameSaveData object from current game state. Serializes to JSON via JsonUtility.ToJson(). Writes to saveFilePath.	IOException: logs error; does not crash game.
GameSaveData Load()	saveFilePath may or may not exist.	If file exists, reads text, deserializes via JsonUtility.FromJson<GameSaveData>(). Returns populated GameSaveData. If file not found, returns null (triggers new game).	IOException: returns null, logs warning.
GameSaveData Fields	Type	Description	
savedDay	int	Day number at save time.	
inventoryItems	List<ItemData>	Serializable representation of inventory contents.	
npcRelationships	Dictionary<int, NPCStatsData>	NPC ID → stats snapshot for all NPCs.	
cropStates	List<CropPlotData>	State of each crop plot (crop type, growth stage, watered flag).	
playerMoney	int	Player's coin balance.	
playerMental	int	Player's mental energy value.	

3.2 AI & Dialogue Package

3.2.1 LLMDialogueSystem

LLMDialogueSystem is the orchestrator for all AI-driven NPC conversations. It constructs system prompts, manages the dialogue FSM, calls LLMApiClient asynchronously, parses responses, and triggers stat updates. One instance exists per NPC (attached as a component on each NPC prefab), sharing a single LLMApiClient that is injected by NPCManager at startup.

Fields:

Field	Type	Access	Description
apiClient	LLMApiClient	private	Injected reference to the shared API client. Set by NPCManager.Initialize().
currentState	DialogueState	private	FSM state enum: {Idle, InputListening, Processing, ResponseGeneration, Termination}.
conversationHistory	List<string>	private	Rolling list of prior exchanges in this session, formatted as 'Player: ...' and 'NPC: ...'. Used to build context window.
maxHistoryLength	int	private const	Maximum number of history entries to include in each prompt. Default: 10. Prevents token overflow.
fallbackMessages	string[]	private static	Pre-written fallback strings returned on API failure. E.g., 'Sorry, I didn't catch that. Can you ask again?'
ownerNPC	NPC	private	Reference to the NPC component this system belongs to. Used to access npcID, name, role, and NPCStats.

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
async Task<string> GenerateNPCResponse(NPC npc, string playerMessage)	apiClient != null. npc != null. playerMessage is non-empty string. currentState == Idle or InputListening.	Sets currentState = Processing. Calls FormatPrompt() and apiClient.SendPrompt(). On success: extracts keywords, applies stat changes, appends to conversationHistory, sets currentState = ResponseGeneration, returns response string. On failure: returns random fallback, sets currentState = Idle.	ApiException → caught internally; fallback returned. ArgumentNullException if npc or playerMessage is null.
List<string> ExtractKeywords(string response)	response is a non-null string.	Returns list of recognized stat-change tokens found in response. Tokens match pattern '[Stat+N]' or '[Stat-N]' where Stat ∈ {Love, Trust, Friendship, Mental}.	None; returns empty list if no keywords found.

void ApplyStatChanges(NPC npc, List<string> keywords)	npc != null. keywords may be empty.	For each keyword, calls npc.npcStats.ModifyStat(stat, delta). Fires OnStatChanged event so UIManager can update bars.	None.
string FormatPrompt(NPC npc, string context, string playerMessage)	npc.role and npc.name are set.	Concatenates: system instruction + NPC persona (name, role, traits) + context (day, time of day, relationship summary) + conversation history (last maxHistoryLength entries) + player message.	None; returns well-formed prompt string.
void SetState(DialogueState newState)	None.	Sets currentState = newState. Logs transition for debugging.	None.

3.2.2 LLMApiClient

LLMApiClient is a thin HTTP client responsible solely for the HTTPS communication with the LLM provider. It is provider-agnostic by design: the endpoint and model identifier are configurable, enabling a swap from DeepSeek to Gemini without changing other classes.

Fields:

Field	Type	Access	Description
apiKey	string	private	Secret API key. Read from environment variable or Unity StreamingAssets at runtime. Never hard-coded.
endpoint	string	private	Full HTTPS URL of the chat completions endpoint. E.g., 'https://api.deepseek.com/v1/chat/completions'.
modelIdentifier	string	private	Model string passed in the request body. E.g., 'deepseek-chat' or 'gemini-pro'.
timeoutSeconds	float	private const	HTTP request timeout. Default: 8.0f. If the API has not responded within this window, the request is aborted and a fallback is triggered.

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
async Task<string> SendPrompt(string prompt)	apiKey and endpoint are set. prompt is non-null.	Constructs JSON body with model, messages array (system + user roles). Sends HTTP POST via UnityWebRequest with HTTPS. Awaits response. Parses response JSON to extract content string. Returns NPC reply.	ApiException (4xx/5xx): thrown to caller. TimeoutException: thrown if response exceeds timeoutSeconds. JsonParseException: thrown if response body is malformed.

string FormatPrompt(NPC npc, string context, string message)	npc fields populated.	Builds the messages array JSON fragment. Combines system prompt (persona + rules) with user message. Returns serialized JSON string.	None.
--	-----------------------	--	-------

3.2.3 NPCManager

NPCManager maintains the roster of all active NPCs and routes player messages to the correct NPC. It is referenced by GameManager and by InputManager for interaction triggering.

Fields:

Field	Type	Access	Description
npcs	List<NPC>	public	All NPC instances in the current scene. Populated by FindObjectsOfType<NPC>() in Awake().
sharedApiClient	LLMApiClient	private	Single LLMApiClient instance shared across all NPCs to avoid key duplication.

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
void Awake()	Scene loaded.	Populates npcs list. Instantiates sharedApiClient. Injects sharedApiClient into each NPC's LLMDialogueSystem.	NullReferenceException if an NPC prefab is missing LLMDialogueSystem component — logged, NPC skipped.
void UpdateNPCs()	Called by GameManager each frame if needed.	Iterates npcs, calls NPC.PerformRoleAction() for any autonomous NPC behavior (e.g., wandering).	None.
void OnDayEnd()	Called by GameManager.EndDay().	Resets per-day conversation counters on all NPCs. Restores NPC positions to default.	None.
NPC GetNPCById(int npcID)	npcID is valid.	Returns NPC from npcs where npc.npcID == npcID. Returns null if not found.	None; null returned on no match.

3.2.4 NPC

NPC is a MonoBehaviour attached to each NPC prefab. It stores the character's identity, current dialogue FSM state, and runtime context. It delegates LLM communication to its LLMDialogueSystem component.

Fields:

Field	Type	Access	Description
-------	------	--------	-------------

npcID	int	public	Unique identifier. Assigned in Unity Inspector.
npcName	string	public	Display name, e.g., 'Granny Mae'. Used in prompts and UI.
role	string	public	NPC archetype string, e.g., 'shopkeeper', 'farmer', 'herbalist'. Injected into LLM system prompt.
dialogueContext	string	public	Static personality description and world lore relevant to this NPC. E.g., 'You are a warm elderly shopkeeper who loves gossip and knows everyone in the village.'
npcStats	NPCStats	public	Reference to the NPCStats value-object. Composed on Awake.
dialogueSystem	LLMDialogueSystem	private	Reference to the sibling LLMDialogueSystem component.
conversationSlotsUsed	int	private	Number of conversations initiated today. Resets on OnDayEnd. Max = 3 per day.

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
async Task ReceivePlayerMessage(string message)	message is non-null. conversationSlotsUsed < 3.	Increments conversationSlotsUsed. Calls dialogueSystem.GenerateNPCResponse(this, message). On completion, raises OnNPCResponded(response) event.	Slotexhaus OnConvers event; does
void UpdateRelationshipStats(List<string> keywords)	keywords list from ExtractKeywords.	Delegates to npcStats.ModifyStat() for each keyword token.	None.
void PerformRoleAction()	Called by NPCManager.UpdateNPCs().	Executes idle behavior based on role (e.g., shopkeeper stays behind counter, farmer wanders field).	None.

3.2.5 NPCStats

NPCStats is a serializable value-object (not a MonoBehaviour) holding the five relationship dimensions for one NPC. It exposes a unified ModifyStat() method to minimize conditional logic in callers.

Fields:

Field	Type	Access	Description
love	int	public	Affection towards the player. Range [0, 100]. Default: 50.
trust	int	public	Player's perceived honesty. Range [0, 100]. Default: 50.
friendship	int	public	General social bond. Range [0, 100]. Default: 50.
mental	int	public	NPC's current mood / energy. Range [0, 100]. Default: 75.

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
void ModifyStat(string statName, int delta)	statName $\in$ {'love', 'trust', 'friendship', 'mental'}. delta $\in$ [-10, 10].	Clamps the target stat to [0, 100] after applying delta. Fires OnStatChanged event.	ArgumentException if statName is unrecognized — logs warning, no change applied.
int GetStat(string statName)	statName $\in$ valid set.	Returns current value of named stat.	ArgumentException if unrecognized.

3.3 Player & Interaction Package

3.3.1 Player

Player is the MonoBehaviour script on the player character. It acts as the central coordinator for player-side state: position, movement, inventory access, NPC interactions, farming actions, and sleep.

Fields:

Field	Type	Access	Description
position	Vector2	public	Current world-space position. Updated by Move().
moveSpeed	float	private	Units per second. Default: 4.0f.
inventory	Inventory	public	Composed Inventory component on the same GameObject.
stats	PlayerStats	public	Composed PlayerStats component on the same GameObject.
nearbyNPC	NPC	private	The NPC currently within interaction radius, or null. Updated by trigger collider events.
nearbyPlot	CropPlot	private	The CropPlot currently within interaction radius, or null.

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
void Move(Vector2 direction)	GameManager.currentState == Playing.	Translates character by direction * moveSpeed * Time.deltaTime. Updates position field.	None.
void InteractWithNPC(NPC npc)	npc != null. nearbyNPC == npc. stats.mental > 0.	Opens DialogueUI. Calls UIManager.ShowDialogue(npc). Subsequent player input handled by InputManager → NPC.ReceivePlayerMessage().	stats.mental == 0 OnMentalExhaust interaction blocked
void PlantCrop(CropType type, CropPlot plot)	inventory.HasItem(SeedItem(type)). plot.crop == null.	Calls farmingSystem.PlantCrop(type, plot). Removes seed from inventory.	Inventory.ItemNo if seed not present occur if pre-condit
void WaterCrop(CropPlot plot)	plot != null. plot.crop != null. !plot.isWatered.	Calls farmingSystem.WaterCrop(plot).	None.
void Sleep()	GameManager.currentState == Playing.	Calls GameManager.EndDay(). Resets stats.mental to 100. Triggers fade-to-black UI transition.	None.
void OnTriggerEnter2D(Collider2D other)	Collider tagged 'NPC' or 'CropPlot'.	Sets nearbyNPC or nearbyPlot. Fires UIManager.ShowInteractionPrompt().	None.
void OnTriggerExit2D(Collider2D other)	Collider was nearbyNPC or nearbyPlot.	Clears nearbyNPC or nearbyPlot. Hides interaction prompt.	None.

3.3.2 InputManager

InputManager captures raw keyboard and mouse events each frame and raises high-level action events that other systems subscribe to. It acts as the sole adapter between Unity's Input system and game logic.

Fields:

Field	Type	Access	Description
OnMoveInput	Action<Vector2>	public event	Fired every frame when WASD / arrow keys are held. Payload is normalized direction vector.
OnInteractPressed	Action	public event	Fired when the interact key (E) is pressed.
OnSleepPressed	Action	public event	Fired when the sleep key (Z) is pressed.
OnSubmitDialogue	Action<string>	public event	Fired when player hits Enter in the dialogue input box. Payload is the input text.

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
string GetPlayerTextInput()	Called by DialogueUI when submit event fires.	Returns current text in the chat input TMP_InputField. Clears the field.	None.
bool DetectInteraction()	Called each frame by Update().	Returns true if E key is down this frame.	None.
void Update()	MonoBehaviour active; GameManager.currentState == Playing.	Reads all relevant Input.GetKey / GetKeyDown. Fires appropriate events.	None.

3.3.3 Inventory

Inventory is a serializable container attached to the Player. It holds Item objects and exposes safe add/remove/query operations.

Fields:

Field	Type	Access	Description
items	List<Item>	private	The backing collection. Exposed via read-only property Items.
maxSlots	int	private const	Hard capacity limit. Default: 20 slots.

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
bool AddItem(Item item)	item != null.	If items.Count < maxSlots: adds item, fires OnInventoryChanged, returns true. Otherwise returns false.	ArgumentNullException if item is null.

bool RemoveItem(Item item)	item != null.	Removes first occurrence from items. Fires OnInventoryChanged. Returns true on success, false if not found.	ArgumentNullException if item is null.
bool HasItem(Item item)	item != null.	Returns items.Any(i => i.itemID == item.itemID).	None.
List<Item> GetAllItems()	None.	Returns shallow copy of items list.	None.

3.3.4 PlayerStats

PlayerStats holds the player-side numeric attributes. Unlike NPCStats, PlayerStats has only two fields — mental (energy) and money — since emotional attributes belong to NPCs.

Fields:

Field	Type	Access	Description
mental	int	public	Current mental energy. Range [0, 100]. Depleted by conversations (−5 per exchange). Restored to 100 on Sleep().
money	int	public	In-game coin balance. Default: 50 coins at game start.

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
void ModifyMental(int amount)	None.	mental = Mathf.Clamp(mental + amount, 0, 100). Fires OnMentalChanged event.	None.
void ModifyMoney(int amount)	None.	money = Mathf.Max(money + amount, 0). Fires OnMoneyChanged event.	None.

3.3.5 Item

Item is a simple serializable data class (not MonoBehaviour) representing any tradeable or usable object.

Field	Type	Access	Description
itemID	int	public	Unique identifier. Used for equality comparison.
itemName	string	public	Display name. E.g., 'Tomato Seed', 'Fresh Carrot'.
value	int	public	Base sell price in coins.
itemType	ItemType	public	Enum {Seed, Produce, Gift, Currency}.

3.4 Economy & Environment Package

3.4.1 FarmingSystem

FarmingSystem manages the grid of CropPlots and the crop lifecycle. It is the authoritative source for all farming state transitions.

Fields:

Field	Type	Access	Description
cropPlots	CropPlot[]	private	Array of all CropPlot objects in the scene. Populated by FindObjectsOfType<CropPlot>() in Awake().

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
void PlantCrop(CropType type, CropPlot plot)	plot.crop == null. player inventory has matching seed.	Instantiates Crop data-object (type, growthStage=0, growthTime per CropType lookup). Assigns to plot.crop. Sets plot.isWatered = false. Fires OnCropPlanted event.	InvalidOperationException if plot already occupied.
void WaterCrop(CropPlot plot)	plot.crop != null. !plot.isWatered.	Sets plot.isWatered = true. Updates crop sprite to show watered state via UIManager.	None.
void AdvanceGrowth()	Called by GameManager.EndDay().	Iterates all cropPlots. For each: if isWatered && crop != null: crop.growthStage++. If growthStage >= crop.growthTime: marks harvestable. Resets all plot.isWatered = false.	None.
Item HarvestCrop(CropPlot plot)	plot.crop != null. crop.growthStage >= crop.growthTime.	Creates Item(itemID=crop.cropType, itemName=..., value=crop.sellPrice). Clears plot.crop = null. Returns Item for caller to add to inventory.	InvalidOperationException if crop not yet mature.
void UpdateCrops()	Called each frame if animated sprites are used.	Updates crop visual stage based on growthStage (sprite swap).	None.

3.4.2 CropPlot

CropPlot is a MonoBehaviour on each farm-tile GameObject. It maintains the state of a single growing slot.

Fields:

Field	Type	Access	Description
crop	Crop	public	The Crop currently growing here. Null if empty.
isWatered	bool	public	True if watered this day. Reset by FarmingSystem.AdvanceGrowth().
growthStage	int	public	Current growth stage (0 = just planted, growthTime = harvestable). Mirrors crop.growthStage for UI access.

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
void Grow()	Called by FarmingSystem.AdvanceGrowth(). crop != null. isWatered == true.	Increments crop.growthStage. Updates growthStage field.	None.

3.4.3 Crop

Crop is a plain serializable data-object representing one plant. It is not a MonoBehaviour.

Field	Type	Access	Description
cropType	string	public	One of: 'Tomato', 'Carrot', 'Wheat'.
growthTime	int	public	Days required to reach harvestable state. Tomato=3, Carrot=2, Wheat=4.
growthStage	int	public	Current progress towards growthTime.
sellPrice	int	public	Base coin value when harvested. Tomato=10, Carrot=8, Wheat=6.

3.4.4 TradingSystem

TradingSystem manages the shop economy. It supports both static and LLM-dynamic pricing and executes transactions with proper inventory and money mutations.

Fields:

Field	Type	Access	Description
prices	Dictionary<string, int>	public	Base price map: itemName → coin value. Loaded from ScriptableObject at startup.
dialogueSystem	LLMDialogueSystem	private	Reference to the shopkeeper NPC's dialogue system, used for optional dynamic pricing.

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
bool BuyItem(Player player, Item item)	prices.ContainsKey(item.itemName). player.stats.money >= prices[item.itemName].	Deducts price from player.stats.money via ModifyMoney(-price). Adds item to player.inventory. Fires OnTransactionComplete(item, 'buy'). Returns true.	Returns false and calls UIManager.ShowNotification('enough coins.') if insufficient funds.

<code>bool SellItem(Player player, Item item)</code>	<code>player.inventory.HasItem(item).</code>	Calculates salePrice via <code>GetPrice(item, player.stats)</code> . Removes item from inventory. Calls <code>player.stats.ModifyMoney(+salePrice)</code> . Fires <code>OnTransactionComplete</code> . Returns true.	Returns false if item not in inventory.
<code>async Task<int> GetPrice(Item item, PlayerStats playerStats)</code>	<code>item != null.</code>	If shopkeeper friendship ≥ 75 : queries LLM for dynamic price (via <code>LLMApiClient.SendPrompt</code>). Otherwise returns <code>prices[item.itemName]</code> . Clamps result to <code>[base * 0.8, base * 1.5]</code> .	<code>ApiException</code> : falls back to base price + logs warning.

3.5 UI Package

3.5.1 UIManager

UIManager is the single facade for all UI state changes. Other packages call UIManager methods rather than manipulating Unity UI components directly, ensuring a clean separation between game logic and presentation.

Fields:

Field	Type	Access	Description
dialogueUI	DialogueUI	private	Reference to the DialogueUI sub-component managing the chat window.
statBars	Dictionary<string, Slider>	private	Name → Unity Slider mapping for Love, Trust, Friendship, Mental bars.
moneyText	TMP_Text	private	TextMeshPro element displaying player coin balance.
notificationText	TMP_Text	private	TMP element for transient on-screen messages (e.g., 'Crop planted!'). Auto-fades after 3 seconds.
shopPanel	GameObject	private	Root GameObject of the shop overlay. Toggled by ShowShop() / HideShop().
interactionPrompt	GameObject	private	Small label shown above nearby interactable objects.

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
void UpdateInventoryUI()	Called after any inventory change.	Re-renders all item slots in the inventory panel with current contents.	None.
void ShowDialogue(NPC npc, string text)	dialogueUI != null.	Calls dialogueUI.DisplayNPCResponse(npc.npcName, text).	None.
void ShowNotification(string message)	message is non-null.	Sets notificationText.text = message. Starts fade coroutine.	None.
void UpdateStatBars(NPC npc)	npc.npcStats != null.	Updates each Slider in statBars dictionary from npc.npcStats values.	KeyNotFoundException if a stat bar is missing from dictionary — logs warning.
void ShowShop()	shopPanel != null.	Sets shopPanel.SetActive(true). Pauses player movement input.	None.
void HideShop()	shopPanel != null.	Sets shopPanel.SetActive(false). Restores player movement input.	None.
void ShowInteractionPrompt(string label)	interactionPrompt != null.	Sets prompt text and SetActive(true).	None.
void HideInteractionPrompt()	interactionPrompt != null.	Sets interactionPrompt.SetActive(false).	None.

3.5.2 DialogueUI

DialogueUI manages the chat window sub-panel: history scrolling, NPC response display, the player input field, and the 'Thinking...' indicator shown during API latency.

Fields:

Field	Type	Access	Description
history	List<string>	private	Complete transcript of the current conversation session. Rendered in the scroll view.
currentText	string	private	The NPC's most recently received response. Exposed for typewriter rendering.
scrollView	ScrollRect	private	Unity ScrollRect component containing the chat history.
inputField	TMP_InputField	private	Player text entry field.
thinkingIndicator	GameObject	private	'Thinking...' spinner/label shown while API call is in progress.

Methods:

Signature	Pre-conditions	Post-conditions	Exceptions
void DisplayNPCResponse(string npcName, string text)	scrollView is active.	Appends '[npcName]: [text]' to history. Instantiates new chat bubble UI element in scroll view. Scrolls to bottom. Starts typewriter coroutine.	None.
void CapturePlayerInput()	inputField != null.	Reads inputField.text. Clears field. Fires OnPlayerInputSubmitted(text) event for InputManager / NPC to consume.	None.
IEnumerator TypewriterEffect(string fullText, TMP_Text target)	target != null.	Reveals fullText character by character with configurable delay (default 0.03s per char). Fires OnTypewriterComplete when done.	None.
void ShowThinkingIndicator(bool show)	thinkingIndicator != null.	Sets thinkingIndicator.SetActive(show).	None.
void ClearHistory()	None.	Clears history list. Destroys all child elements in scroll view content.	None.

4. Glossary

Term	Definition
Asynchronous Coroutine	A Unity programming pattern using IEnumerator or async/await that allows code execution to be suspended and resumed without blocking the main game loop thread.
API Key	A secret string credential issued by an LLM cloud provider (DeepSeek, Gemini) used to authenticate HTTP requests. Must never be committed to source control.
Chat Completions Endpoint	The specific HTTPS route on an LLM provider's REST API that accepts a messages array and returns a generated continuation. Standardized by OpenAI's API format, also adopted by DeepSeek.
Clamp	The mathematical operation <code>Mathf.Clamp(value, min, max)</code> that constrains a number to a specified range, used for all stat field mutations.
Composition	A strong UML ownership relationship where the owned object's lifecycle is tied to the owner (e.g., Player owns Inventory — if the Player object is destroyed, the Inventory is too).
Content Window / Context Window	The maximum number of tokens an LLM can process in a single prompt. LLM Valley manages this by limiting conversationHistory to <code>maxHistoryLength = 10</code> entries.
CropType	An enumeration {Tomato, Carrot, Wheat} identifying the species of a crop.
DontDestroyOnLoad	A Unity API call that marks a GameObject to survive scene transitions, used by GameManager for the singleton pattern.
Enum / Enumeration	A C# value type whose values are named constants. Used for GameState, DialogueState, CropType, ItemType.
Event (C#)	A language construct (public event <code>Action<T></code>) providing a publisher-subscriber mechanism without direct coupling between classes.
Fallback Response	A pre-written NPC dialogue line served when the LLM API fails or times out. Preserves gameplay flow without crashing.
FindObjectsOfType<T>()	A Unity API method that searches the scene hierarchy for all active GameObjects with component T. Used in Awake() methods to populate reference lists.
FSM (Finite State Machine)	A model consisting of a finite set of states, transitions between states triggered by inputs or events, and associated actions. Governs NPC dialogue flow.
GameObject	The fundamental entity in Unity. Can hold any number of MonoBehaviour components that define its behavior.
GetComponent<T>()	Unity API to retrieve a component of type T attached to the same GameObject or a specified object.
IEnumerator	The C# interface returned by Unity coroutine methods. Allows yielded execution over multiple frames.
JSON (JavaScript Object Notation)	A text-based data serialization format. Used both for API payloads (LLM requests/responses) and for the local save file.

JsonUtility	Unity's built-in JSON serialization class. Only serializes public fields; does not support Dictionary natively (requires wrapper).
Keyword Token	A structured string embedded by the LLM in its response following a prompt instruction, e.g., '[Trust+1]'. Parsed by ExtractKeywords() to modify NPCStats.
LLM (Large Language Model)	A neural network trained on vast text corpora capable of generating coherent, context-appropriate text given a prompt. Examples: GPT-4, DeepSeek-chat, Gemini Pro.
Modular Architecture	A design approach where components are decoupled and replaceable. LLM Valley achieves this by abstracting the LLM provider behind LLMApiClient.
MonoBehaviour	Unity's base class for all component scripts. Provides lifecycle hooks: Awake, Start, Update, OnDestroy, and more.
NPC (Non-Player Character)	A game entity controlled by AI rather than the human player.
NPCStats	A serializable value-object holding the five social/emotional dimensions (Love, Trust, Friendship, Mental) for a single NPC.
OnApplicationQuit	A Unity MonoBehaviour callback invoked when the application is closing. Used by SaveSystem to ensure data persistence.
Prefab	A pre-configured, reusable Unity asset representing a complete GameObject hierarchy with all its components and settings.
Prompt Engineering	The practice of carefully crafting LLM input text (the 'prompt') to elicit desired, safe, and contextually appropriate responses.
REST (Representational State Transfer)	An architectural style for networked APIs using standard HTTP methods (GET, POST) and JSON bodies.
ScriptableObject	A Unity data container asset that stores configuration independent of any scene. Used for the price dictionary in TradingSystem.
Singleton	A creational design pattern ensuring a class has at most one instance, with a global access point. Used by GameManager.
System Prompt	The 'system' role message in an LLM chat completion request, providing meta-instructions and persona context the model should follow for all subsequent user messages.
TMP_Text / TextMeshPro	Unity's high-quality text rendering library. Used for all on-screen labels, chat history, and stat displays.
Token	The atomic unit of text processed by an LLM. Approximately 3/4 of a word. API pricing is typically per-token.
Typewriter Effect	A UI animation that reveals text character-by-character to simulate typing, improving NPC response readability and pacing.
UnityWebRequest	Unity's built-in HTTP client class used by LLMApiClient to send POST requests to the LLM API endpoint.
Value Object	A design pattern for objects defined by their values rather than identity. Crop, Item, NPCStats, and PlayerStats are value objects.
Vector2	A Unity struct representing a 2D (x, y) coordinate or direction. Used for player position and movement.

5. References

- [1] TED University, Faculty of Engineering, "LLM Valley Project Proposal," Team 21, October 31, 2025.
- [2] TED University, Faculty of Engineering, "LLM Valley Project Analysis Report," Team 21, November 28, 2025.
- [3] TED University, Faculty of Engineering, "LLM Valley Project Specifications Report," Team 21, November 14, 2025.
- [4] TED University, Faculty of Engineering, "LLM Valley High Level Design Report," Team 21, December 30, 2025.
- [5] B. Bruegge and A. H. Dutoit, Object-Oriented Software Engineering: Using UML, Patterns, and Java, 2nd ed. Prentice-Hall, 2004. ISBN: 0-13-047110-0.
- [6] L. Song, "LLM-Driven NPCs: Cross-Platform Dialogue System for Games and Social Platforms," 2025.
- [7] L. M. Csepregi, "The Effect of Context-aware LLM-based NPC Conversations on Player Engagement in Role-playing Video Games," Aalborg University, Copenhagen, Denmark, 2024.
- [8] G. Zeng, "A review of AI-based game NPCs research," Applied and Computational Engineering, vol. 15, pp. 155–159, 2023.
- [9] Unity Technologies, Unity Manual and Scripting API, 2024. [Online]. Available: <https://docs.unity3d.com/Manual/index.html>
- [10] Unity Technologies, "Using JsonUtility for Saving and Loading Data," Unity Documentation, 2023. [Online]. Available: <https://docs.unity3d.com/ScriptReference/JsonUtility.html>
- [11] IEEE Computer Society, "Software Engineering Code of Ethics and Professional Practice." [Online]. Available: <https://www.computer.org/education/code-of-ethics>
- [12] ACM, "ACM Code of Ethics and Professional Conduct," 2018. [Online]. Available: <https://www.acm.org/code-of-ethics>
- [13] ConcernedApe, Stardew Valley [Video game], ConcernedApe, 2016. Available: https://store.steampowered.com/app/413150/Stardew_Valley/
- [14] Sam Edwards, Travellers Rest [Video game], Isolated Games, 2020. Available: https://store.steampowered.com/app/1139980/Travellers_Rest/
- [15] LLM Valley Project Website. Available: <https://llmvalley.framer.website/>
- [16] DeepSeek, "DeepSeek API Documentation," 2024. [Online]. Available: <https://api-docs.deepseek.com/>
- [17] Google, "Gemini API Developer Documentation," 2024. [Online]. Available: <https://ai.google.dev/gemini-api/docs>